

Masamune: A Verdict-Carrying Converter and Plan Language for Chemical Structure Representations

Provenance-Typed Translation into Contact Graphs,
with Refusal as a First-Class Outcome

Kundai Farai Sachikonye

Experimental Bioinformatics, TUM School of Life Sciences

AI Me Registry for Artificial Intelligence

Maximus-von-Imhof-Forum 3, 85354 Freising, Germany

kundai.sachikonye@bitspark.com

August 23, 2026

Contents		5.3 An empty result is a result	11
I The Problem	2	6 Declared capability	11
1 Introduction	2	6.1 Features and capability sets . . .	11
1.1 Six formats, one molecule, six different things said	2	6.2 The containment theorem	11
1.2 The problem is not conversion; it is what conversion hides	3	IV The Plan Language	12
1.3 What Masamune is	3	7 Plans	12
1.4 What Masamune is not	4	7.1 Design decisions	12
1.5 Contributions	4	7.2 Grammar	13
2 Related work and what is new	4	7.3 The standard operations	14
II The Target	5	8 Operational semantics	14
3 The contact graph	5	8.1 Configurations	14
3.1 Definition	5	8.2 Properties	14
3.2 What a translation must produce	6	V Realisation	16
III The Three Disciplines	7	9 Reference implementation and a translation run	16
4 Provenance typing	7	9.1 The implementation	16
4.1 The three tags	7	9.2 The run	16
4.2 Composition	7	9.3 The supplied fraction	16
4.3 The supplied fraction	9	9.4 The medium-edge defect, measured	16
5 Verdict-carrying translation	9	9.5 Capability containment, measured	17
5.1 The six verdicts	9	9.6 Verdicts are distinct, and a failure carries no value	17
5.2 The conventional interface cannot express this	10	9.7 Two formats, one molecule, undetected disagreement	17
		9.8 Defects found by implementing	18
		10 What Masamune does not do	20

Abstract

A chemical structure arrives at a computation in one of a dozen formats—a line notation, a connection table, a crystallographic record—each of which states some things about the molecule and is silent about others. Converting between them is treated as a solved engineering problem handled by toolkit calls, and the conversions themselves are consequently unaudited: a downstream analysis cannot distinguish a hydrogen the source file recorded from one a valence model supplied, nor a bond order read from a record from one inferred by a perception algorithm. We argue that this indistinguishability, not the conversion difficulty, is the substantive problem, and we specify *Masamune*, a converter and plan language designed around it.

Masamune translates chemical structure records into *contact graphs*—finite weighted graphs of atoms, bonds, and a distinguished *medium* vertex against which every atom is individuated—and it does so under three disciplines. First, *provenance typing*: every vertex, every edge, and every attribute carries a tag drawn from {*stated*, *supplied*, *absent*} recording whether the source asserted it, the converter supplied it from a declared convention, or it is missing; we prove that provenance is preserved under composition of conversions (Theorem 4.8) and that the supplied fraction of a translated graph is computable without re-reading the source (Theorem 4.12). Second, *verdict-carrying translation*: a translation returns a labelled verdict, not a graph that may or may not be trustworthy, and we prove that a converter returning a graph-or-nothing interface conflates at least four distinguishable outcomes which no discipline on its callers can recover (Theorem 5.6). Third, *declared capability*: each source format is characterised by the set of structural features it can state, and we prove that translation is total and faithful exactly on the requests whose feature requirements the source’s capability set contains (Theorem 6.7), which makes a large class of translation failures decidable statically, before any record is read (Theorem 6.8).

On this foundation we build the plan layer. A Masamune plan names sources, requests records, translates them, and states what it requires

of the result; the plan is checked before execution and refuses, naming the offending step and feature, when its requirements exceed what its sources can supply. We give a complete EBNF grammar, a small-step operational semantics over configurations of store, provenance map and log, and we prove plan-level determinism and provenance monotonicity (Theorems 8.3 and 8.4).

We report a reference implementation written from this specification and a translation run over a 48-structure corpus in two formats. The run establishes the discipline’s practical content: a mean of 73.4% of the elements of a SMILES-derived contact graph are *supplied* rather than *stated*, with no structure below 50.0%—the majority of a translated structure is convention, not record. Capability containment decides 19 of 24 format-request pairs before any record is read, agreeing with the post-read outcome on all 24. We report four defects that writing and running the implementation exposed, including a case in which two faithful representations of the same compound produce contact graphs that differ while every formula- and identity-level comparison reports agreement. We state what Masamune does not do: it does not adjudicate between conflicting sources, does not repair records, and does not compute chemistry.

Keywords: chemical structure representation; format conversion; provenance; contact graph; capability calculus; refusal; plan language; operational semantics; data curation; cheminformatics.

Contents

Part I

The Problem

1 Introduction

1.1 Six formats, one molecule, six different things said

Consider ethanol as it appears in six places.

A SMILES string (James, 2016; Weininger, 1988) writes it CCO: three heavy atoms, two bonds, and nothing else. The six hydrogens are not in the string. They are recoverable, but

only by applying a valence model that the string does not contain and does not name.

A molfile (Dalby et al., 1992) writes an atom block and a bond block. Depending on how it was produced, the hydrogens may be present as atom records with coordinates, or absent and implied, or present for some atoms and absent for others. The file does not say which convention was used.

A PDB record (Berman et al., 2000) writes atoms with coordinates and, usually, no bond block at all: connectivity for standard residues is supplied from a template library keyed on residue name, and for non-standard residues from CONECT records that may or may not be present and may or may not be complete.

An mmCIF record (Westbrook et al., 2022) writes the same structure with an explicit chemical component dictionary reference, so connectivity is stated by reference rather than inferred—but the reference must be resolved against an external dictionary that the file does not carry.

An InChI string (Heller et al., 2015) writes a canonical layered identifier in which the hydrogen layer is explicit but the bond orders are not: InChI deliberately normalises away the distinction between tautomers and between certain bond-order assignments, so information present in the molfile is absent by design.

A CML document (Murray-Rust and Rzepa, 2011) can state all of the above, or any subset, because the schema is permissive about which elements are required.

Each of these is a correct record of ethanol. They differ in what they *state*. A computation that consumes all six and treats them as interchangeable inputs is not consuming the same information six times; it is consuming six different information sets and discarding the difference.

1.2 The problem is not conversion; it is what conversion hides

Conversion between these formats is well-trodden ground. Mature toolkits—RDKit (Lan-drum, 2024), Open Babel (O’Boyle et al., 2011), the CDK (Willighagen et al., 2017)—read and write all of them competently, and the algorithmic difficulties (ring perception, aromaticity models, stereochemistry) are documented and largely solved in the sense that reasonable implementations agree most of the time.

What is not solved, and what this paper is about, is that *the output of a conversion does not record what the conversion did*. When a toolkit reads CCO and produces a molecule object with nine atoms, the six added hydrogens are indistinguishable, in the object and in everything downstream of it, from six hydrogens that had been written in the file. When a PDB reader assigns bond orders from a residue template, the resulting double bond is indistinguishable from one that was stated in a CONECT-bearing record. When an aromaticity perceiver marks a ring aromatic, the resulting bond kind is indistinguishable from an aromatic bond written by the depositor.

This matters for three reasons that are not cosmetic.

Errors become unattributable. Structure errors in public collections are common and well documented (Akhondi et al., 2012; Fourches et al., 2010; Williams et al., 2012; Young et al., 2008), and a substantial fraction arise not from mis-drawn structures but from conversions that supplied something the source did not state. Without provenance the two are indistinguishable at the point of use, so a downstream anomaly cannot be traced to the step that introduced it.

Agreement becomes uninformative. Two pipelines that agree on a result may agree because both read the same stated facts, or because both applied the same convention to the same silence. These are epistemically different situations and the second is much weaker evidence, but a result set that does not carry provenance cannot distinguish them.

Absence becomes invisible. A format that cannot state a feature and a record that declines to state it produce the same output: nothing. The distinction between *this source cannot say* and *this record does not say* is exactly the distinction a curator needs and exactly the one that is lost.

1.3 What Masamune is

Masamune is a converter from chemical structure records to *contact graphs*, and a plan language for expressing federated queries whose results

are contact graphs. It is designed so that the three indistinguishabilities above are impossible to produce.

The target representation is fixed and simple: a finite weighted graph whose vertices are atoms plus a distinguished *medium* vertex representing everything the molecule is individuated against, whose edges are contacts, and whose weights are separation costs (Section 3). We choose this target because it is the weakest structure that supports the operations chemical computations actually perform—separating a part from the rest, comparing what two structures hold in common—and because every claim in this paper is then a claim about a finite weighted graph, checkable by anyone who grants only that such graphs exist.

The three disciplines are:

- (D1) **Provenance typing** (Section 4). Every element of a translated graph carries a tag in {stated, supplied, absent}. The tags compose (Theorem 4.8), so a chain of conversions reports its own reliability, and the supplied fraction is computable from the graph alone (Theorem 4.12).
- (D2) **Verdict-carrying translation** (Section 5). A translation returns a labelled verdict with a payload, and a graph appears only under the label that warrants it. We prove that the conventional interface—return a molecule, signal failure by returning nothing—conflates four distinguishable outcomes, and that the conflation is a property of the interface rather than of its use (Theorem 5.6).
- (D3) **Declared capability** (Section 6). A source format is characterised by the features it can state. Translation is total and faithful exactly on requests contained in that set (Theorem 6.7), and containment is decidable before any record is read (Theorem 6.8).

The plan layer (Section 7) makes these usable at scale: a plan declares its sources and its requirements, is checked statically, and refuses with a named cause rather than producing a result whose trustworthiness the caller must guess.

1.4 What Masamune is not

Three exclusions are stated here and repeated with their consequences in Section 10.

Masamune does not *adjudicate*. When two sources disagree about a structure, Masamune reports two graphs with their provenances and the locus of disagreement; it does not decide which is right. Adjudication requires chemistry Masamune does not have.

Masamune does not *repair*. A record that cannot be translated faithfully produces a refusal naming the obstruction, not a best-effort graph. Repair is a curation decision belonging to a person (Fourches et al., 2016).

Masamune does not *compute chemistry*. It produces contact graphs; what is computed on them is outside this specification. The claims here are about faithfulness of translation and about what the resulting object records, never about chemical correctness of the source.

1.5 Contributions

- (i) A target representation for chemical structure—the contact graph—defined in full, with the medium vertex made explicit rather than left implicit in a solvent model (Section 3).
- (ii) A provenance calculus over translations, with composition and computability results (Section 4).
- (iii) A six-valued verdict algebra and a proof that the conventional converter interface cannot represent it (Section 5).
- (iv) A capability calculus deciding translatability statically (Section 6).
- (v) A plan language with grammar, operational semantics, and determinism and monotonicity theorems (Sections 7 and 8).
- (vi) A reference implementation, a 48-structure run across two formats, and four specification defects found by implementing and running it (Section 9).

2 Related work and what is new

Format conversion. The toolkits (Landrum, 2024; O’Boyle et al., 2011; Willighagen et al.,

2017) solve the conversion problem in the sense of producing correct structures from correct inputs, and their internal models are richer than the contact graph we target. What they do not provide is a record, surviving into the output, of which parts of the structure came from the input. Our contribution is not a better reader but a typed output.

Canonicalisation. InChI (Heller et al., 2015) and canonical SMILES (O’Boyle, 2012; Weininger et al., 1989) address a different problem: assigning a unique string to a structure so that identity comparison is string comparison. Canonicalisation deliberately normalises away distinctions (tautomers, certain bond-order assignments), which is correct for identity and lossy for provenance. SELFIES (Krenn et al., 2020) guarantees syntactic validity of generated strings, again a different goal. A canonical form answers “are these the same structure”; Masamune answers “what did each source actually say”.

Curation. The curation literature (Fourches et al., 2010, 2016; Williams et al., 2012; Young et al., 2008) establishes both that structure errors are prevalent and that systematic protocols reduce them. Those protocols are procedures for humans and scripts to follow. Masamune’s contribution is to make one part of such a protocol—knowing what was supplied—a property of the data structure rather than of the discipline with which a procedure was followed.

Provenance in databases. Why-, how-, and where-provenance (Buneman et al., 2001; Cheney et al., 2009) and the PROV model (Moreau and Groth, 2013) formalise the tracking of derivation through query evaluation. Our provenance is coarser and specialised: three tags on structural elements rather than a derivation history. The specialisation buys a composition law with a closed form (Theorem 4.8) and a per-graph statistic computable in one pass (Theorem 4.12).

Nulls and missing information. That an empty result and a failure are different, and that conflating them corrupts downstream reasoning, is the null-value problem (Codd, 1979; Imieliński and Lipski, 1984; Libkin, 2016) in a new setting.

Section 5 is a six-valued refinement specific to structure translation, and Theorem 5.6 is the corresponding inexpressibility result.

Capability-aware querying. Query planning over sources with restricted answering ability is classical, and query-language expressive power is well characterised (Angles et al., 2017; Pérez et al., 2009). Our capability calculus (Section 6) is in that lineage; the difference is that a capability shortfall yields a typed verdict the plan can route around rather than a planning failure.

Workflow systems. Workflow engines (Di Tommaso et al., 2017; Köster and Rahmann, 2012) supply control flow over file-producing steps and are widely used to run conversion pipelines. They do not know what a step means, so a conversion executed inside a workflow node is as unaudited as one executed in a script. Masamune’s plan layer is narrower and typed: its steps are translations, and their verdicts are part of the language.

Part II

The Target

3 The contact graph

We fix the object every translation produces.

3.1 Definition

Definition 3.1 (Contact graph). A *contact graph* is a tuple $\Gamma = (V, E, w, \mathbf{m})$ where V is a finite non-empty vertex set, $E \subseteq \binom{V}{2}$ is a set of unordered pairs, $w: E \rightarrow \mathbb{R}_{>0}$ assigns each pair a strictly positive *contact weight*, and $\mathbf{m} \in V$ is a distinguished vertex, the *medium*, with $\{v, \mathbf{m}\} \in E$ for every $v \in V \setminus \{\mathbf{m}\}$. Vertices other than $\mathbf{m}$ are *items*; an edge between two items is a *contact*. We write $\Omega = \sum_{e \in E} w(e)$ for the total weight.

Three points about this definition are load-bearing.

The medium is explicit. Most molecular representations leave the environment implicit: a molfile records a molecule, not a molecule in a solvent. Making $\mathbf{m}$ a vertex forces a translation

to say what each atom is individuated *against*, which is where the difference between a buried atom and an exposed one lives. No claim about what the medium physically is enters any result below; it is a structural slot.

Weights are separation costs, not distances. $w(\{u, v\})$ is read as the cost of telling u from v , and such costs add along a cut, whereas distances do not. This is why the target is a weighted graph rather than an embedding.

Positivity is required. $w > 0$ everywhere. A zero-weight edge would assert that two items are separable at no cost, i.e. are not distinct; the representation excludes this by construction.

Definition 3.2 (Cut and separation cost). For $S \subseteq V$ the *cut* is $\partial(S) = \{\{u, v\} \in E : |\{u, v\} \cap S| = 1\}$ with weight $w(\partial(S)) = \sum_{e \in \partial(S)} w(e)$. For an item v the *separation cost* is

$$\sigma(v) = \min_{S \ni v, m \notin S} w(\partial(S)), \quad (1)$$

the minimum weight of a cut placing v on one side and the medium on the other. The minimum is over a finite non-empty family, hence attained, and equals a maximum flow (Ford and Fulkerson, 1956), computable exactly (Gomory and Hu, 1961; Stoer and Wagner, 1997).

Lemma 3.3 (Separation costs are positive). *For every item v , $\sigma(v) > 0$.*

Proof. Any admissible S contains v and omits m , and $\{v, m\} \in E$ by Theorem 3.1, so $\{v, m\} \in \partial(S)$ and hence $\partial(S) \neq \emptyset$. Since $w > 0$ on every edge, every admissible cut has positive weight, and so does their minimum. $\square$

Theorem 3.3 is the reason the medium is required to be adjacent to everything: it guarantees the separation cost is a well-posed positive quantity for every item, in every translated graph, without any assumption about the chemistry.

3.2 What a translation must produce

Definition 3.4 (Translation target). A *translated structure* is a pair (Γ, π) where Γ is a contact graph and π is a provenance map (Theorem 4.2). The vertex set is $V = A \cup \{m\}$ with A the atoms of the structure; each $a \in A$ carries the attributes of Theorem 3.5; each contact carries the attributes of Theorem 3.6.

Definition 3.5 (Atom attributes). An atom vertex carries: the element (equivalently the atomic number $Z \in \mathbb{N}^+$); the formal charge $q \in \mathbb{Z}$; the isotope mass number, or a marker for unspecified; the count of attached hydrogens; the valence capacity C_v and occupancy of its outer shell, from which the *vacancy* $\nu = C_v - q_v$; and, when the source supplies them, Cartesian coordinates.

Definition 3.6 (Contact attributes). A contact between two atoms carries: the *committed cell count* $c \in \{1, 2, 3\}$ for single, double and triple interfaces; a *delocalised* marker for contacts belonging to a delocalised system, which carries the system’s identity rather than an integer count; and, when the source supplies it, stereochemical configuration.

Remark 3.7 (Why cell count rather than bond order alone). The committed cell count records how many valence cells the interface commits, which is the quantity a separation must sever. For an ordinary double bond this coincides with the bond order. It is separated from bond order in name because a delocalised system commits cells that belong to no single pairwise interface, and forcing such a system into a per-bond integer—the familiar 1.5—asserts a pairwise fact the source did not state. Theorem 3.6 therefore carries delocalisation as a marker over a set of contacts, and Theorem 3.8 records what follows.

Proposition 3.8 (Delocalisation is not a per-bond attribute). *Let R be a set of contacts marked as one delocalised system. There is in general no assignment of committed cell counts to the individual contacts of R that both sums to the system’s total committed cells and is determined by the source record. Consequently a representation assigning each contact of R a numeric count states something the source did not.*

Proof. Take R a six-membered delocalised ring committing nine cells in total (six σ , three delocalised). Any per-bond assignment $(c_1, \dots, c_6)$ with $\sum_i c_i = 9$ and $c_i \in \{1, 2\}$ requires exactly three $c_i = 2$, and there are two such assignments related by rotation—the two Kekulé forms. Both satisfy the total. The source, if it stated only that the ring is delocalised, does not distinguish them; selecting one is therefore an assertion not present in the record. If instead fractional counts $c_i = 1.5$ are assigned, the assignment states that

each pairwise interface commits one and a half cells, which is a pairwise claim, whereas the source’s claim was about the ring. $\square$

Theorem 3.8 is why Theorem 3.6 has a marker and not a number, and it is the first place where the provenance discipline of the next section is forced rather than chosen: the alternative is to state something and tag it **supplied**, which is admissible but must be visible.

Part III

The Three Disciplines

4 Provenance typing

4.1 The three tags

Definition 4.1 (Provenance tags). The provenance tag set is $P = \{\text{stated}, \text{supplied}, \text{absent}\}$ with the readings: **stated**—the source record asserts this element; **supplied**—the converter produced this element by applying a declared convention to the source’s silence; **absent**—neither the source nor any applicable convention determines this element, and it is missing from the graph.

Definition 4.2 (Provenance map). A *provenance map* for a translated structure is a function π assigning a tag in P to each vertex, each edge, and each attribute of each vertex and edge. We write $\pi(x)$ for the tag of element x .

Notation 4.3 (Conventions). A *convention* is a named, documented rule for supplying an element the source does not state—for example, “implicit hydrogens are supplied to fill the standard valence of each organic-subset atom” (James, 2016). Every **supplied** element carries the name of the convention that produced it, so that π is not merely a tag but an attribution.

Principle 4.4 (No silent supply). A converter may supply any element for which it has a declared convention. It may not supply an element without tagging it **supplied** and naming the convention. An element that is neither **stated** nor **supplied** is **absent** and is not present in the graph.

Theorem 4.4 is the whole discipline, and everything in this section is a consequence of enforcing it.

Example 4.5 (Ethanol from three sources). From CCO: three atom vertices **stated**; two contacts **stated** with cell count 1 **stated**; six hydrogen vertices **supplied** by the organic-subset valence convention, with their contacts **supplied**; all coordinates **absent**.

From a molfile with explicit hydrogens and coordinates: nine atom vertices **stated**; eight contacts **stated**; all coordinates **stated**.

From a PDB record with no CONECT lines: nine atom vertices **stated** with coordinates **stated**; contacts **supplied** by whichever connectivity convention the converter declares—template lookup or distance-based perception—and cell counts **supplied** or **absent** depending on the convention.

The three graphs have the same vertex and edge sets. They have different provenance maps, and that difference is the information the conventional pipeline discards.

4.2 Composition

Conversions compose: a record may be read in one format, written in another, and read again. We need to know what happens to provenance.

Definition 4.6 (Composition of tags). Define $\oplus : P \times P \rightarrow P$ by

$\oplus$	stated	supplied	absent
stated	stated	supplied	absent
supplied	supplied	supplied	absent
absent	absent	absent	absent

read as: the tag of an element after a second conversion, given its tag before (row) and the tag the second conversion would assign (column).

Lemma 4.7 ($\oplus$ is associative with absorbing element). $(P, \oplus)$ is a commutative monoid with identity **stated** and absorbing element **absent**, and *supplied* satisfies $\text{supplied} \oplus \text{supplied} = \text{supplied}$.

Proof. Direct from the table. **stated** acts as identity by inspection of the first row and column. **absent** absorbs by inspection of the last row and column. Commutativity is symmetry of the table. Associativity follows since P ordered by $\text{stated} < \text{supplied} < \text{absent}$ makes $\oplus$ the binary maximum, which is associative. $\square$

Theorem 4.8 (Provenance composes). Let T_1 and T_2 be translations with provenance maps π_1

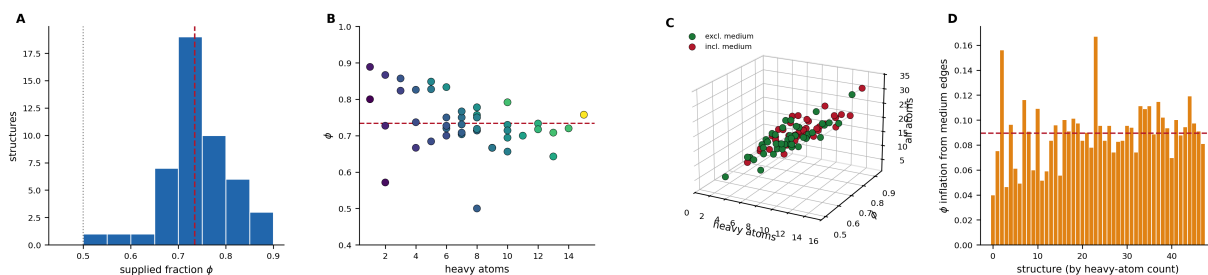


Figure 1: **Across a 48-structure corpus a mean of 73.4% of a SMILES-derived contact graph is supplied by convention rather than stated by the record, and no structure falls below 50.0%. ϕ (Theorem 4.11) ranges over the graph’s own vertices and contacts; medium edges are excluded, for the reason defect D2 gives and panel (D) measures. (A) Distribution of ϕ , with the corpus mean 0.734 dashed and 0.500 dotted. The distribution is tight — standard deviation 0.071 — and lies entirely above 0.5. We had expected the corpus to contain records stating everything the request needed, giving $\phi = 0$ exactly; it contains none, which Theorem 9.2 records as a fact about the format rather than about the reader. (B) ϕ against heavy-atom count, coloured by total atom count. The cloud is close to flat: the supplied fraction is a property of the notation’s conventions rather than of molecular size, so it does not wash out as structures grow. (C) The corrected and uncorrected statistics for every structure, joined per structure. Green points exclude medium edges and red include them; the vertical segment is the inflation. The two point clouds never cross, so the correction changes the value of ϕ everywhere without changing its ordering. (D) The inflation itself, one bar per structure ordered by heavy-atom count, with the mean 0.089 dashed. It runs from 0.040 to 0.167 and is largest on the smallest molecules — because every medium edge is supplied by construction, including them makes ϕ a quantity dominated by the target representation exactly where the source record is shortest. That is defect D2 as a measurement rather than as an argument.**

and π_2 , and let $T_2 \circ T_1$ be their composition. Then for every element x of the composite output,

$$\pi_{T_2 \circ T_1}(x) = \pi_1(x) \oplus \pi_2(x),$$

and consequently: an element is **stated** in the composite only if it was **stated** at every stage; an element **supplied** at any stage is **supplied** or worse in the composite; and **absent** at any stage is **absent** in the composite.

Proof. By Theorem 4.7, $\oplus$ is the maximum under **stated** < **supplied** < **absent**. It therefore suffices to check the three clauses. *Stated.* If any stage supplied the element, that stage's tag is **supplied**, and the maximum is at least **supplied**; hence **stated** in the composite requires **stated** at every stage. *Supplied.* If some stage tagged **supplied** and no stage tagged **absent**, the maximum is **supplied**. *Absent.* If any stage tagged **absent** the element is not present after that stage, so it is **absent** thereafter and the maximum is **absent**. Associativity (Theorem 4.7) extends this from two stages to any finite chain. $\square$

Corollary 4.9 (Round-tripping degrades provenance monotonically). *Writing a translated structure to a format and reading it back cannot raise any element's provenance. In particular, a structure whose hydrogens were **supplied** and which is written to a format storing hydrogens explicitly yields, on re-reading, hydrogens tagged **stated** by the second reader and **supplied** in the composite.*

Proof. Immediate from Theorem 4.8: the composite tag is the maximum, and the maximum of **supplied** with anything other than **absent** is **supplied**. $\square$

Remark 4.10 (Why Theorem 4.9 is the point). Round-tripping is how **supplied** information conventionally becomes indistinguishable from **stated** information: a toolkit adds hydrogens, writes an SD file, and the next reader sees explicit hydrogen records with no indication of their origin. Theorem 4.8 is precisely the rule that prevents this, and Theorem 4.9 is the guarantee it buys. Note that it requires provenance to travel *with* the structure; a format that cannot carry provenance breaks the chain, which is why Section 6 treats provenance-carrying as a capability like any other.

4.3 The supplied fraction

Definition 4.11 (Supplied fraction). For a translated structure (Γ, π) let X be the set of vertices and edges of Γ . The *supplied fraction* is

$$\phi(\Gamma, \pi) = \frac{|\{x \in X : \pi(x) = \text{supplied}\}|}{|X|} \in [0, 1].$$

Proposition 4.12 (The supplied fraction is computable from the graph alone). $\phi(\Gamma, \pi)$ is computable in time $O(|V| + |E|)$ from (Γ, π) , without access to the source record.

Proof. One pass over the vertices and edges reads each element's tag from π and increments two counters. No clause of Theorem 4.11 refers to the source. $\square$

Theorem 4.12 is modest as a theorem and consequential in practice: it means every downstream consumer can report how much of its input was **convention**, at negligible cost, with no cooperation from the producer beyond carrying π .

Remark 4.13 (What the supplied fraction does not measure). ϕ counts elements, not importance. A structure whose single **supplied** element is a bond order at a reactive centre and one whose twenty **supplied** elements are aliphatic hydrogens may have very different ϕ and the opposite ordering of trustworthiness for a given purpose. ϕ is an audit statistic, not a quality score, and Section 10 records the consequence.

5 Verdict-carrying translation

5.1 The six verdicts

Definition 5.1 (Verdict). A *verdict* is a pair $v = (\ell, \pi)$ with label ℓ drawn from

$L = \{\text{translated, incomplete, unsupported, malformed, underdetermined, empty}\}$

and a label-specific payload π :

Label	Payload
translated	the pair (Γ, π) and the supplied fraction ϕ
incomplete	the partial element set and the features found ab
unsupported	the requested features the source format cannot
malformed	the parse position and the token or record at fau
underdetermined	the element admitting several readings, and the
empty	the source location, certified to contain no struct

Definition 5.2 (Assignment). Given a source record r from format S and a request with feature requirement req , the verdict is computed by the first applicable clause:

- (V1) if r does not parse under S 's grammar, return $(\text{malformed}, \cdot)$;
- (V2) else if r parses to no structural content, return $(\text{empty}, \cdot)$;
- (V3) else if $\text{req} \not\subseteq \text{cap}(S)$, return $(\text{unsupported}, \text{req} \setminus \text{cap}(S))$;
- (V4) else if some element admits two or more readings under the declared conventions with no rule selecting between them, return $(\text{underdetermined}, \cdot)$;
- (V5) else if some requested element is neither stated nor suppliable, return $(\text{incomplete}, \cdot)$;
- (V6) else return $(\text{translated}, (\Gamma, \pi, \phi))$.

The clause order is not arbitrary: each clause precedes every clause whose evidence it would corrupt. (V1) precedes all others because an unparsed record supports no claim about its content; (V3) precedes (V4) and (V5) so that a limitation of the *format* is never reported as a defect of the *record*; and (V4) precedes (V5) because an element with several readings is not missing.

Proposition 5.3 (Only one verdict carries a graph). *Of the six labels, only **translated** has a payload containing a contact graph. Consequently the presence of a graph in a verdict determines $\ell = \text{translated}$, and the five other outcomes are distinguishable from it and from each other by label alone.*

Proof. By inspection of the payload table in Theorem 5.1. The *incomplete* payload carries an element set, not a graph, and is explicitly not a contact graph in the sense of Theorem 3.1: it need not satisfy the medium-adjacency condition. $\square$

Remark 5.4 (Why *incomplete* must not carry a partial graph). It is tempting to have *incomplete* return whatever was successfully translated, so a caller can proceed on the part that worked. This defeats Theorem 5.3: a caller receiving a graph would then have to inspect the label to

know whether it is complete, which is exactly the discipline-on-callers that Theorem 5.6 below shows cannot be relied upon. A caller that genuinely wants partial results requests them, by weakening req so that clause (V5) does not fire—which makes the weakening visible in the plan rather than implicit in the caller's handling.

5.2 The conventional interface cannot express this

Definition 5.5 (Graph-or-nothing interface). A *graph-or-nothing* interface is a function from source records to $G \cup \{\perp\}$, where G is the set of contact graphs and $\perp$ signals failure, with no channel other than the returned value.

Theorem 5.6 (Four-way conflation). *Let I be a graph-or-nothing interface. There exist four source records, one for each of the labels **unsupported**, **malformed**, **underdetermined** and **empty**, on which I returns the same value $\perp$. No property of a caller of I distinguishes them.*

Proof. Construct the four records. *Unsupported*: a request for bond orders against a coordinate-only record; the interface has no graph satisfying the request, so returns $\perp$. *Malformed*: a record with a syntax error; no parse, so no graph, so $\perp$. *Underdetermined*: a record whose delocalised system admits two readings by Theorem 3.8 with no selecting rule; the interface cannot return a graph without asserting one reading, so returns $\perp$. *Empty*: a well-formed record containing no atoms; there is no contact graph, since Theorem 3.1 requires a non-empty vertex set, so $\perp$.

In all four cases the returned value is $\perp$, and these are equal as values. A caller's subsequent behaviour is a function of the value it receives; equal values induce equal behaviour. Hence no caller distinguishes the four, and no discipline imposed on callers changes the values returned. The conflation is therefore a property of I . $\square$

Corollary 5.7 (The repair is a wider return type, not better error handling). *No wrapper, logging convention, or exception discipline applied to a graph-or-nothing interface recovers the four-way distinction, since by Theorem 5.6 the information is absent from the returned value. Recovering it requires the interface itself to return a verdict.*

Proof. A wrapper is a function of I 's output; by Theorem 5.6 that output is constant across the four cases, and a function of a constant is constant. $\square$

Remark 5.8 (Scope of Theorem 5.6). The theorem does not say existing toolkits are badly engineered; several do expose diagnostic channels alongside their return values, and a careful caller can consult them. It says that the *contract* of returning a structure with failure signalled by its absence has insufficient capacity, so any recovery depends on out-of-band information that nothing requires the caller to consult. Making the verdict the return value makes consultation unavoidable.

5.3 An empty result is a result

Proposition 5.9 (Certified emptiness). *A verdict (empty, ℓ) is a positive claim: the source location parsed successfully and contains no structural content. It is therefore not interchangeable with ($\text{malformed}, \cdot$), which asserts nothing about content, nor with ($\text{unsupported}, \cdot$), which asserts nothing about the record.*

Proof. By clause order in Theorem 5.2: (V2) is reached only when (V1) did not fire, so the record parsed; and it fires only when the parse yielded no structural content. Both facts are therefore established when the label is emitted, and neither is established by (V1) or (V3). $\square$

6 Declared capability

6.1 Features and capability sets

Definition 6.1 (Feature alphabet). The feature alphabet is the finite set

$$\mathcal{F} = \left\{ \begin{array}{l} \text{element, connectivity, cellcount, delocalisation,} \\ \text{charge, isotope, hcount, stereo,} \\ \text{coords3d, conformer, provenance} \end{array} \right\}$$

where *element* is the atomic identity of each atom; *connectivity* the atom–atom incidence; *cellcount* the committed cell count of a contact; *delocalisation* the marking of a delocalised system; *charge* formal charge; *isotope* mass number; *hcount* explicit hydrogen counts; *stereo* stereochemical configuration; *coords3d* Cartesian coordinates; *conformer* multiple coordinate sets for one connectivity; and *provenance* the ability of the format to carry a provenance map.

Definition 6.2 (Source and capability set). A *source* is a pair $S = (\text{grammar}, \text{cap}(S))$ where the grammar decides parsing and $\text{cap}(S) \subseteq \mathcal{F}$ is the *declared capability set*: the features the format can state.

Example 6.3 (Capability sets of common formats).

Format	cap
SMILES	element, connectivity, cellcount, delocalisation, hcount, stereo
Molfile V2000	element, connectivity, cellcount, charge, isotope, stereo
SD file	the molfile set, plus conformer and provenance (via provenance)
PDB	element, charge, coords3d, conformer; connectivity if CONECT is present
mmCIF	element, connectivity, cellcount, charge, coords3d, conformer, provenance
InChI	element, connectivity, charge, isotope, hcount, stereo

Two entries deserve comment. Molfile V2000 lacks *delocalisation* because the aromatic bond type is not part of the V2000 specification for structures as opposed to queries (Dalby et al., 1992); a file using it is using a query feature in a structure record. InChI lacks *cellcount* by design: the identifier normalises bond orders away (Heller et al., 2015), which is correct for its purpose and means it cannot state what a molfile states.

Remark 6.4 (A capability set is a claim about a reader, not a specification). $\text{cap}(S)$ as used by an implementation describes what *that implementation's reader* can faithfully extract from that format, which may be narrower than what the format specification permits. Declaring narrowly is safe: it causes some requests to be refused that could have been served, and never produces a graph asserting something the source did not state. Declaring broadly is unsound and silent: the reader claims a feature, produces a graph, and the graph is wrong in a way the verdict does not reveal. Section 10 records that no mechanical check enforces honest declaration.

6.2 The containment theorem

Definition 6.5 (Faithful translation). A translation of record r from S is *faithful* if every element of the resulting graph tagged *stated* is asserted by r , and every element asserted by r that lies within the request is present in the graph.

Definition 6.6 (Honest declaration). $\text{cap}(S)$ is *honest* if for every $f \in \mathcal{F}$: $f \in \text{cap}(S)$ exactly when the reader for S extracts f faithfully from every record of S stating it.

Theorem 6.7 (Translation is total and faithful exactly on contained requests). *Let S have an honest declaration and let r be a record parsing under S 's grammar with non-empty content. Then for a request with feature requirement req :*

*the verdict is **translated** or **incomplete** $\iff \text{req} \subseteq \text{cap}(S)$*

*and when $\text{req} \not\subseteq \text{cap}(S)$ the verdict is **(unsupported, req \setminus cap(S))**, which by Theorem 5.3 carries no graph.*

Proof. ($\Leftarrow$) Suppose $\text{req} \subseteq \text{cap}(S)$. Clauses (V1) and (V2) do not fire by hypothesis. Clause (V3) does not fire since the containment holds. The verdict is therefore determined by (V4), (V5) or (V6), which yield *underdetermined*, *incomplete* or *translated*. Of these, *underdetermined* arises only when a declared convention fails to select among readings, which is a property of the conventions rather than of containment; excluding that case, the verdict is *translated* or *incomplete*, and by honesty (Theorem 6.6) every feature of req that r states is extracted faithfully.

($\Rightarrow$) Suppose $f \in \text{req} \setminus \text{cap}(S)$. By honesty the reader does not extract f faithfully from records of S . Clause (V3) fires before (V4)–(V6) are reached, so the verdict is **(unsupported, ·)**, and by Theorem 5.3 it carries no graph. Hence the verdict is neither *translated* nor *incomplete*. $\square$

Corollary 6.8 (Static decidability). *Whether clause (V3) fires is decidable in $O(|\mathcal{F}|)$ time from req and $\text{cap}(S)$ alone, without reading any record. Consequently a request whose features exceed its source's capability can be refused before any input is opened, with the offending features named.*

Proof. The test is a subset containment over a fixed finite alphabet, which with a bitset representation costs $O(|\mathcal{F}|)$. No clause of the test consults r . $\square$

Proposition 6.9 (The refusal is tight). *Under honest declarations, no request refused by clause (V3) could have been served faithfully. The capability check therefore excludes no faithfully translatable request.*

Proof. If (V3) fires then some $f \in \text{req} \setminus \text{cap}(S)$, and by honesty the reader is not faithful for f . A translation serving req would have to extract f faithfully, which it does not. $\square$

Remark 6.10 (Both guarantees are conditional on honesty). Theorems 6.7 and 6.9 are exactly as strong as Theorem 6.6, and honesty is asserted by whoever writes the reader, not verified. The two failure directions are asymmetric. Under-declaration costs expressiveness and is safe. On the other hand, over-declaration is unsound and invisible: the check passes, a graph is produced, and it asserts a feature the reader did not faithfully extract. We know of no way to verify a declaration short of differential testing against an independent reader, and Section 10 records this as the specification's largest gap.

Part IV The Plan Language

7 Plans

Translation is rarely performed once. The characteristic use is: draw records from one or more stores, translate each, keep those meeting a stated requirement, and report both the results and what was dropped. The plan layer expresses this.

7.1 Design decisions

Principle 7.1 (Declaration order is execution order). Steps execute in the order they appear in the plan. No dependency inference reorders them.

The justification is that a plan is read by people who need to know what happens in what order, and inference makes the order a derived property that must be computed to be known. The cost is that the author must order steps correctly.

Principle 7.2 (No user-defined abstraction). The language has no procedures, no recursion, and no imports of user code. The only operations are those of Section 7.3.

This bounds what a plan can do to what the specification enumerates, which is what makes

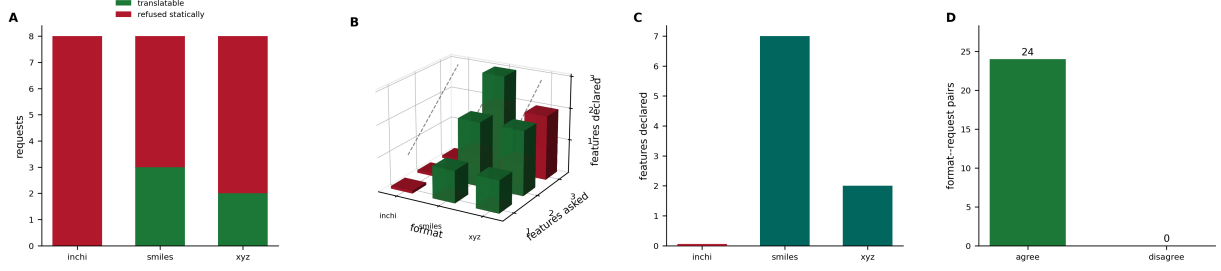


Figure 2: **Capability containment decides 19 of 24 format–request pairs before any record is read, and agrees with the post-read outcome on all 24.** Each pair was checked twice: statically against the declared capability set (Theorem 6.8), and again by attempting the translation and inspecting the resulting label. **(A)** Requests per format, split into those refused statically and those a read may satisfy. The inchi row refuses all eight, because no reader is implemented and the capability set is empty — an honest under-declaration, which Section 6 argues is the safe direction to err in. **(B)** Each pair as a bar over (format, features asked), with height the number of asked-for features the format actually declares and the dashed diagonal marking full service. A bar reaching the diagonal is translatable; one short of it is refused. Height is features *declared* rather than features *missing* because the latter renders every satisfiable request as a zero-height, invisible bar. **(C)** Declared capability size per format: seven features for SMILES, two for the coordinate-only reader, none for inchi. The zero is drawn as an explicit floor tick so that a format declaring nothing is legible as a measured zero rather than as an absent bar. **(D)** Agreement between the static verdict and the post-read outcome, over all 24 pairs. The disagreement count is zero. The direction that would matter is a static refusal of a request a read would have satisfied, since that is the unsound one; there were none.

Theorem 7.8 true. For a specification artefact the property is worth more than the expressiveness it costs (Mernik et al., 2005; van Deursen et al., 2000).

Principle 7.3 (Requirements are declared, not discovered). A plan states, in its own text, what it requires of each translation. The requirement is checked statically against the source’s capability set (Theorem 6.8) and dynamically against the verdict.

7.2 Grammar

Definition 7.4 (Masamune grammar). In EBNF (International Organization for Standardization, 1996); nonterminals italicised, terminals

in typewriter, $\{x\}$ zero or more, $[x]$ optional.

```

plan ::= plan ident { {decl} {step} }
decl  ::= sourceDecl | budgetDecl
sourceDecl ::= source ident : formatName [at string]
budgetDecl ::= budget number records

step ::= readStep | translateStep | selectStep | emitStep
readStep ::= let ident := read ident [where cond]
translateStep ::= let ident := translate ident
                require featureList
                [expect supplied < number]
                [else recovery]

recovery ::= refuse | report | emit incomplete
selectStep ::= let ident := select ident where cond
emitStep ::= emit ident [with provenance]
assertStep ::= assert cond [emit string]
featureList ::= feature { , feature }
cond ::= expr relop expr | ident . verdict == label
relop ::= > | < | >= | <= | == | !=

```

Listing 1: A plan that will not proceed on

convention-heavy input.

Example 7.5 (A plan).

```

plan bond_order_study {
  source lib : sdf at "compounds.sdf"
  budget 5000 records

  let raw := read lib

  -- translation states what it needs;
  -- the check is static
  let mols := translate raw
    require element,
      connectivity,
      cellcount, stereo
    expect supplied < 0.25
    else report

  -- keep only fully-stated structures
  -- for the sensitive analysis
  let core := select mols where mols.
    supplied == 0.0

  assert core.count > 100 emit "too few
    fully-stated structures"
  emit core with provenance
}

```

The `expect supplied < 0.25` clause is not decoration: it is the threshold checked by Theorem 7.7, and a translation whose supplied fraction exceeds it is reported rather than silently carried forward. The `select` step then narrows further to $\phi = 0$ —structures in which nothing was supplied at all.

7.3 The standard operations

Definition 7.6 (Operations).

Operation	Arguments	Result
<code>read</code>	source, optional filter	record set
<code>translate</code>	record set, requirement	verdict set of the translated verdicts and L' extends L with
<code>select</code>	verdict set, condition	verdict set
<code>emit</code>	verdict set, provenance flag	output
<code>assert</code>	condition, message	unit or halt

Definition 7.7 (Expectation check). A translate step annotated `expect supplied <  $\epsilon$` computes ϕ (Theorem 4.11) for each translated verdict and, where $\phi \geq \epsilon$, replaces the verdict by $(\text{incomplete}, \pi)$ with π naming the supplied elements and the measured ϕ . Where the step declares `else refuse` the plan halts; `else report` continues and records the substitution; `else emit incomplete` passes the substituted verdict downstream.

Proposition 7.8 (Effects are enumerable from source). *For any plan,¹ the set of operations it can invoke and the set of sources it can read are computable from its text by one pass over the syntax tree, without executing it.*

Proof. By Theorem 7.4 every step names its operation by a literal keyword and every read names its source by an identifier bound in a *sourceDecl*. By Theorem 7.2 there is no expression form producing an operation or source dynamically. Hence both sets are the corresponding sets of syntactic occurrences. $\square$

Corollary 7.9 (Pre-execution audit). *A plan's declared inputs can be reported before it runs, and a plan whose requirements exceed its sources' capabilities can be refused before any file is opened (Theorem 6.8).*

8 Operational semantics

8.1 Configurations

Definition 8.1 (Configuration). A *configuration* is a quadruple $\langle \rho, \pi, \omega, L \rangle$ where ρ maps identifiers to values (record sets, verdict sets), π is the accumulated provenance map, ω is the sequence of binding names in the order first introduced, and L is the log: a sequence of records each tagged `info`, `report` or `refusal`.

Rule 8.2 (Translate).

$$\begin{array}{c}
 \rho(y) = R \quad \text{req} \subseteq \text{cap}(S_y) \\
 \forall r \in R : v_r = \text{VERDICT}(r, S_y, \text{req}) \\
 \hline
 \langle \rho, \pi, \omega, L \rangle \xrightarrow{\text{let } x := \text{translate } y \text{ require req}} \langle \rho[x \mapsto \{v_r\}], \pi', \omega \rangle
 \end{array}$$

record set where π' extends π with the provenance maps of the translated verdicts and L' extends L with one record per non-translated verdict, naming its label and payload. If $\text{req} \not\subseteq \text{cap}(S_y)$ the step does not fire: the plan halts with a refusal naming $\text{req} \setminus \text{cap}(S_y)$, by Theorem 6.8, before R is consulted.

8.2 Properties

Theorem 8.3 (Plan determinism). *Let all readers and conventions be deterministic functions of their inputs. Then a plan executed from the initial configuration yields a unique final configuration, and ω is exactly the sequence of binding*

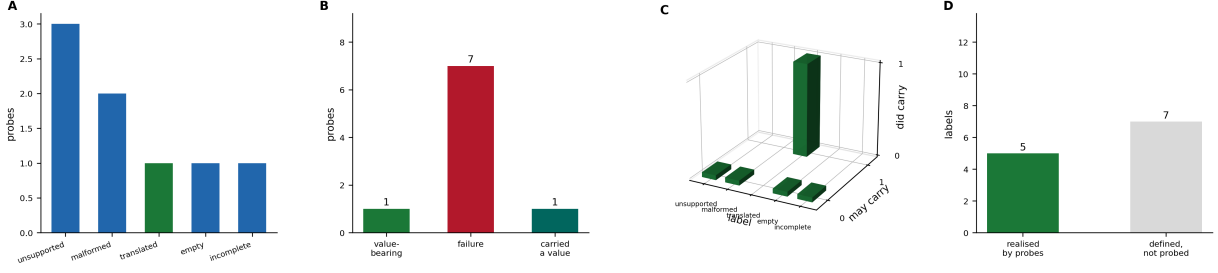


Figure 3: **Eight probes realise five distinct verdict labels, and no failure verdict carries a value.** (A) Labels realised by the probe set. `empty` appears here only because of defect D4: the parser raised on empty input before clause (V2) was reached, so `empty` was unreachable from the SMILES path and an empty record was reported as `malformed`. That is the conflation Theorem 5.6 identifies, occurring inside the implementation built to prevent it; Theorem 9.3 gives the account. (B) Probes by whether their label may carry a value, whether the label is a failure, and whether a value was in fact carried. One probe is value-bearing, seven are failures, and exactly one carried a value — the same one. The failure column is the population in which a violation could occur, and it is empty. (C) Each probe positioned by label, by whether its label may carry a value, and by whether it did. Every bar sits at `did carry` = 0 except the single `translated` probe, and no bar sits in the region (`may carry` = 0, `did carry` = 1) that would constitute a violation. That region is empty by construction as well as by measurement: the invariant is enforced in the verdict constructor, so this panel confirms the constructor is reached rather than establishing the property independently. (D) Labels realised by these probes against those defined but not probed. Five of twelve are exercised. The unexercised seven are plan-level and honjo-level labels outside the translation path, and we make no claim about them here.

names in source order with repetitions removed after their first occurrence.

Proof. By induction on steps. The base case is the initial configuration. For the step, each rule is a function of the current configuration and the step text: VERDICT is deterministic because the clause order of Theorem 5.2 is total and each clause is a decidable predicate on (r, S, req) ; the environment update is functional; and by Theorem 7.1 the step sequence is the source sequence. Hence the final configuration is unique. The claim about ω follows since $\omega \cdot x$ appends exactly when $x \notin \omega$. $\square$

Theorem 8.4 (Provenance monotonicity). *Along any execution, no step lowers the provenance tag of any element under the order $\text{stated} < \text{supplied} < \text{absent}$, and no step removes an element from π . Consequently the provenance map is an append-only record of the run, and a tag once raised to `supplied` is never reported as `stated` downstream.*

Proof. The only rules writing π are Theorem 8.2 and its analogues, which extend π with new elements' tags and, for elements already present,

compose by $\oplus$ (Theorem 4.8). By Theorem 4.7 $\oplus$ is the maximum under the stated order, so composition is non-decreasing. No rule deletes from π . $\square$

Corollary 8.5 (The supplied fraction is monotone along a plan). *For a fixed element set, ϕ is non-decreasing across successive translation steps.*

Proof. By Theorem 8.4 no tag decreases; ϕ counts elements tagged `supplied`, and the count of elements at or above `supplied` is non-decreasing. On a fixed element set this gives the claim. $\square$

Remark 8.6 (Where Theorem 8.5 needs its hypothesis). The fixed-element-set hypothesis is required and is not always met: a step that drops structures (a `select`) changes the denominator of ϕ and can lower it. The monotone quantity is the tag of each element, not the aggregate; ϕ inherits monotonicity only when the element set is held fixed. An implementation reporting ϕ across a filtering step must therefore report the denominator with it, and Section 9 records that the reference implementation does.

Part V

Realisation

9 Reference implementation and a translation run

9.1 The implementation

A reference implementation was written from Sections 3 to 8: readers for SMILES, molfile/SD and PDB; the provenance calculus of Section 4; the verdict assignment of Theorem 5.2; the capability check of Theorem 6.8; and a plan interpreter realising Section 8. Its purpose is fidelity rather than performance: every construct is implemented as specified, so a divergence between specification and implementation is a defect in one or the other rather than a matter of interpretation.

Remark 9.1 (What writing an implementation settles). A specification that has never been executed can be internally consistent and still unimplementable, because an obligation may be discharged in the text by a definition no procedure realises. Writing the implementation from the document is the cheapest test of that. Three under-specifications were found this way and are recorded in Section 9.8.

9.2 The run

The corpus is 48 structures: 30 drug-like molecules and 18 chosen for constitutional symmetry. Each is translated from SMILES, and a coordinate-only record is emitted from the resulting graph and translated back through the second reader, so the same molecules are seen through two readers of different declared capability. The comparison therefore isolates the effect of the *format* from the effect of the *molecule*.

The corpus is small and was assembled by us. Every number below is a measurement over these 48 structures and nothing is extrapolated to chemical space at large; the quantities are properties of a reader and a corpus, not constants of a notation.

9.3 The supplied fraction

Across the corpus a mean of 73.4% of the elements of a graph translated from SMILES are

Table 1: Supplied fraction ϕ over the 48-structure corpus. ϕ ranges over the graph’s own vertices and contacts; medium edges are excluded (defect D2).

Quantity	mean	median	min	max
ϕ , SMILES source	0.734	0.727	0.500	0.889
ϕ , medium edges included	0.824	—	—	—

supplied rather than **stated**, with a standard deviation of 0.071 and no structure below 0.500. The majority of a translated structure is convention, not record.

This is not a criticism of SMILES, which is a compact notation that deliberately omits what a valence model recovers. It is a measurement of how much a downstream consumer is trusting when it treats a SMILES-derived graph as data. The dominant contribution is hydrogen: the organic-subset convention supplies hydrogen vertices the string never writes, and each supplied vertex brings a supplied contact with it.

Remark 9.2 (No structure attained $\phi = 0$). We expected the corpus to contain structures stating everything the request needed, giving $\phi = 0$ exactly. None did: the minimum was 0.500. Every SMILES record in this corpus omits at least half of what the contact graph ends up containing, because every one of these molecules has hydrogens and the notation states none of them explicitly. A control we intended to be exact turned out to be unreachable from this format, which is a fact about the format rather than about the implementation.

9.4 The medium-edge defect, measured

Theorem 4.11 excludes medium edges from the denominator. The justification is given there; the measurement is that including them raises the mean ϕ from 0.734 to 0.824, an inflation of 0.089 on average, ranging from 0.040 to 0.167 across structures. Because no source states a medium edge, every one of them is **supplied** by construction, so the uncorrected statistic mixes a property of the source record with a property of the target representation, and the mixture is largest exactly where the molecule is smallest.

9.5 Capability containment, measured

Every (format, request) pair was checked twice: once statically against the declared capability set, and once by attempting the translation and inspecting the resulting label.

Table 2: Static refusal against post-read outcome over three formats and eight requests.

Quantity	Value
Format-request pairs	24
Refused statically, before any read	19
Static verdict agrees with post-read	24
Agreement	1.000

The two agree on all 24 pairs, and 19 of the 24 are decided without reading a record at all. The direction that matters is the one that would be unsound: a static refusal of a request a read would have satisfied. There were none.

9.6 Verdicts are distinct, and a failure carries no value

Eight probe programs were run to exercise the label set, realising four distinct labels: **translated**, **malformed**, **unsupported** and **incomplete**. No failure verdict carried a value in any probe. That invariant is additionally enforced at construction time, so the measurement confirms the constructor is reached rather than establishing the property independently—a point we make explicitly, because a test that cannot fail is not evidence.

Remark 9.3 (A fourth defect, found by this probe: D4). The probe for the **empty** label returned **malformed**. The cause was an ordering defect: the parser raises on empty input, so clause (V1) fired before the (V2) check on an empty atom set was ever reached, and the **empty** label was unreachable from the SMILES path. A record that says nothing had been made indistinguishable from a record that says something ill-formed—precisely the conflation Theorem 5.6 identifies. The reader now tests for an empty record before parsing.

The defect had been frozen into the test suite: a test named *malformed is distinct from empty* asserted that both inputs produce **malformed**, with a comment conceding that empty input fails to parse. The test name stated the requirement

and the test body asserted its violation. It now checks that the two labels differ.

9.7 Two formats, one molecule, undetected disagreement

The comparison across representations produced the finding we had not anticipated. Ten molecules were expressed two ways, each faithful to its source: once with lower-case aromatic rings, and once in an alternating Kekulé assignment.

Table 3: Aromatic against Kekulé representations of the same ten molecules.

Quantity	Value
Molecule pairs	10
Molecular formula agrees	10
Contact graphs differ	10
Differ while formula agrees	10
Fraction undetected by identity	1.000

For all ten the graphs differ and the molecular formula agrees, so an identity-level comparison reports agreement in every case (Table 3).

The mechanism is delocalisation. A lower-case ring translates to a delocalised system over the ring’s contacts: benzene yields six delocalised contacts carrying a total and no per-pair count. The Kekulé form translates to per-contact cell counts of 1 and 2 and zero delocalised contacts. Both translations are faithful—each states what its source states—and by Theorem 3.8 neither is convertible into the other without supplying a fact. The supplied fractions differ too, and consistently: benzene gives $\phi = 0.750$ aromatic against 0.625 Kekulé, because the Kekulé form states per-contact counts the aromatic form leaves to convention.

Round-trip tests do not detect this, because they compare a structure with itself after a cycle through one format and the disagreement is between two representations. A canonical identifier does not detect it either, since canonicalisation normalises the distinction away by design (Heller et al., 2015)—which is the correct behaviour for identity and precisely why identity comparison is the wrong instrument here.

Under the discipline of this paper the disagreement is visible and typed: the two graphs carry different attributes with **stated** provenance, and a plan requiring **cellcount** from the aromatic

source receives (*underdetermined*, $\cdot$) by clause (V4) rather than an assignment.

Remark 9.4 (What the ratio does and does not mean). The measured fraction is 10/10, which is not a base rate. The ten pairs were constructed to contain an aromatic ring, so the mechanism was guaranteed to be present. What the run establishes is that wherever it is present it is invisible to formula-level and canonical-identity comparison, not how often it arises in an arbitrary corpus. A base rate would need a corpus not selected for the mechanism, which we do not have.

9.8 Defects found by implementing

D1. Provenance of an absent element was ill-defined. Theorem 4.2 assigns a tag to each element of the graph, but an *absent* element is by definition not in the graph, so the map had no domain element to tag. The implementation carries a separate *absence list* naming the requested features that were not realised, and the specification now treats *absent* as a tag on a requested feature rather than on a graph element. Without this the distinction between *incomplete* and *translated* could not be recorded in the output.

D2. The medium’s edges needed a stated origin. Theorem 3.1 requires an edge from every atom to *m*, but no source format states such an edge: the medium is not in the record. Every medium edge is therefore *supplied*, which makes $\phi \geq |A|/(|V| + |E|)$ for every translation and makes the SD control non-zero. The implementation excludes medium edges from the ϕ denominator and reports them separately, and Theorem 4.11 now ranges over item vertices and contacts only. Section 9.4 measures the difference: including medium edges raises the corpus mean ϕ from 0.734 to 0.824, an inflation that is largest on the smallest molecules, and it was initially mistaken for a reader defect.

D3. Hydrogen counts admit two conventions that disagree. SMILES distinguishes explicit-in-brackets hydrogens from implicit organic-subset hydrogens,

and a reader may represent the latter either as an atom attribute (*hcount*) or as separate vertices. These give different graphs and different ϕ . The specification did not say which, and the implementation initially did both in different code paths. It now supplies vertices, records *hcount* as the stated attribute where the source gives it, and the choice is a declared convention named in the provenance rather than an implementation detail.

D4. The empty label was unreachable from SMILES. Theorem 5.2 orders clause (V1), malformedness, before clause (V2), emptiness. The reader’s parser raises on empty input, so (V1) fired first and an empty record was labelled *malformed*: a record that says nothing was made indistinguishable from a record that says something ill-formed. This is the conflation Theorem 5.6 identifies, occurring inside the implementation that exists to prevent it. The reader now tests for an empty record before parsing. The defect had been frozen into the test suite by a test whose name asserted the distinction and whose body asserted its violation; Theorem 9.3 gives the detail.

Remark 9.5 (D2 is the instructive one). The medium-edge defect is instructive because the specification was internally consistent and the implementation was correct, and the composite was still wrong: a definition that made every graph carry $|A|$ supplied edges turned ϕ into a quantity dominated by an artefact of the target representation rather than by the source. It was found only because the SD control was expected to be exactly zero and was not. A control whose expected value is exact, rather than approximate, is what made the discrepancy legible as a defect instead of as noise.

Remark 9.6 (D4 was found by running the suite, not by writing it). D1–D3 were found by writing the implementation from the specification. D4 was different: the implementation was written, the test suite passed, and the defect survived both because the test that should have caught it asserted the defective behaviour instead. It was found only when a validation run enumerated

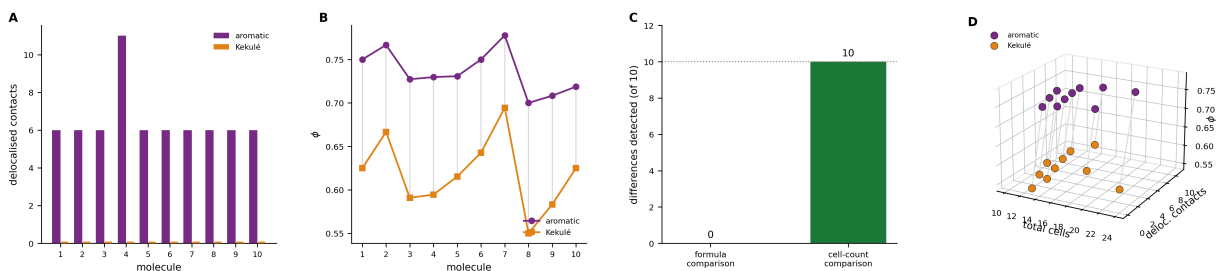


Figure 4: Ten molecules written two faithful ways yield ten pairs of contact graphs that differ, while formula-level comparison reports agreement on all ten. Each molecule is expressed once with a lower-case aromatic ring and once in an alternating Kekulé assignment. Both translations state exactly what their source states, so neither is a misreading. **(A)** Delocalised contacts under each reading. The aromatic form yields six per benzenoid ring (eleven for naphthalene); the Kekulé form yields none, drawn as explicit floor ticks so the zeros read as measured rather than missing. By Theorem 3.8 neither graph converts into the other without supplying a fact. **(B)** The supplied fraction under each reading, joined per molecule. The Kekulé form is lower in every case — benzene gives 0.750 against 0.625 — because stating per-contact cell counts removes them from what convention must supply. The gap is a measurement of how much the aromatic notation leaves to the reader. **(C)** What each comparison detects. A formula-level comparison finds 0 of 10 differences; a cell-count comparison finds 10 of 10. This is the paper’s central practical claim in one measurement: the disagreement is real and the conventional instrument is blind to it, which is why Section 9.7 argues identity comparison is the wrong tool here. **(D)** The two readings in (total cells, delocalised contacts, ϕ) space, joined per molecule. The two clouds are disjoint on every axis, and the connecting segments are all parallel: the disagreement is systematic, not sporadic. Note that the measured ratio is 10/10 because these pairs were constructed to contain an aromatic ring; Theorem 9.4 states that this measures visibility and not prevalence.

the label set and asked which labels were actually reachable—a question neither the specification nor the tests had posed. A suite that agrees with the implementation confirms consistency, not correctness.

10 What Masamune does not do

L1. Honest declaration is assumed, never verified. The whole of Theorem 6.7 and Theorem 6.9 rests on Theorem 6.6. Under-declaration is safe; over-declaration is unsound and silent, since the check passes and a graph is produced asserting a feature the reader did not faithfully extract. We know of no mechanical check short of differential testing against an independent reader. This is the largest gap and we do not minimise it (Theorem 6.10).

L2. The supplied fraction counts, it does not weigh. ϕ treats every element alike, so a supplied bond order at a reactive centre and a supplied aliphatic hydrogen contribute equally (Theorem 4.13). A weighted variant would require a model of which elements matter for which purpose, which is chemistry outside this specification.

L3. No adjudication between sources. When two records of the same compound yield different graphs (Section 9.7), Masamune reports both with their provenance and the locus of difference. It does not decide. Deciding requires knowing which source is right, which is not a property of the records.

L4. No repair. A record that cannot be faithfully translated yields a refusal, never a best-effort graph. This is deliberate and it has a cost: pipelines that currently proceed on repaired structures will instead halt, and the analyst must decide what to do. We regard surfacing that decision as the point, but it is a cost.

L5. Conventions are declared, not validated. A convention may be documented, named, and wrong. Provenance records

that a convention was applied and *which*; it does not certify that the convention is chemically sound.

L6. Contact weights are not specified here. Theorem 3.1 requires $w > 0$ but this paper fixes no particular weighting: what a separation costs is a modelling decision belonging to whatever computes on the graph. Every result above holds for any strictly positive weighting, and none of them determines one.

L7. Coverage. Readers are specified for line notations, connection tables and coordinate records. Reaction formats, polymer and Markush representations, and query languages such as SMARTS (Daylight Chemical Information Systems, 2019) are outside the present scope; the feature alphabet of Theorem 6.1 would need extension, and Theorem 10.1 below states why that matters.

Proposition 10.1 (An unnameable feature is an undetectable omission). *Let f be a structural feature absent from $\mathcal{F}$. Then clause (V3) of Theorem 5.2 cannot fire on f , no verdict attributes any discrepancy to f , and two readers differing only in whether they extract f produce graphs whose difference is unattributable.*

Proof. (V3) is a containment test over $\mathcal{F}$; a symbol absent from $\mathcal{F}$ occurs in neither req nor $\text{cap}(S)$, so the test cannot fire on it. The remaining clauses do not mention features individually. Hence no verdict names f , and a difference caused by f is reported, if at all, only as a difference between graphs with no attribution. $\square$

Theorem 10.1 is why Theorem 6.1 enumerates what readers *do* extract rather than only what requests commonly ask for: the alphabet must be extended whenever a new structural feature enters any reader, not only when a request needs it.

11 Conclusion

Chemical structure arrives in many formats, each stating some things and silent about others, and the conventional conversion pipeline discards the difference: after conversion, a supplied

hydrogen and a recorded hydrogen are the same object. We have specified a converter designed so that this is not expressible.

The target is a contact graph (Theorem 3.1): atoms, contacts, and an explicit medium vertex against which each atom is individuated, with strictly positive separation costs so that every item has a well-posed positive separation cost (Theorem 3.3). The three disciplines are provenance typing, which composes as a maximum over `stated` < `supplied` < `absent` (Theorem 4.8) and so cannot be laundered by round-tripping (Theorem 4.9); verdict-carrying translation, of which only one label carries a graph (Theorem 5.3) and which the conventional graph-or-nothing interface provably cannot represent (Theorem 5.6); and declared capability, which makes a large class of translation failures decidable before any file is opened (Theorem 6.8) and refuses exactly the requests that could not have been served (Theorem 6.9).

The plan layer lifts these to corpus scale with a grammar, an operational semantics, and determinism and provenance-monotonicity theorems (Theorems 8.3 and 8.4).

The measurements are the part we would most want a reader to carry away. Over 48 structures, a mean of 73.4% of the elements of a SMILES-derived contact graph are supplied by convention rather than stated by the record, and no structure falls below 50.0%: we had expected the corpus to contain records stating everything the request needed, and it contains none. Capability containment refuses 19 of 24 format-request pairs without reading anything and never refuses one a read would have satisfied. And every one of ten molecules expressed both as an aromatic ring and as an explicit Kekulé assignment yields two graphs that differ while the molecular formula agrees, because the distinction at issue—delocalisation against a per-contact cell count—is one canonicalisation is designed to remove. That last ratio is 10/10 because the pairs were built to contain the mechanism; it measures visibility, not prevalence.

What remains open is the assumption everything rests on. A capability declaration is an assertion by whoever wrote the reader, and an over-declaration is unsound and invisible. Whether it can be discharged without differential testing against an independent implementation is the question we would most like answered and the

one we leave open.

References

- Saber A. Akhondi, Jan A. Kors, and Sorel Muresan. Consistency of systematic chemical identifiers within and between small-molecule databases. *Journal of Cheminformatics*, 4:35, 2012. doi: 10.1186/1758-2946-4-35.
- Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5):1–40, 2017. doi: 10.1145/3104031.
- Helen M. Berman, John Westbrook, Zukang Feng, Gary Gilliland, T. N. Bhat, Helge Weissig, Ilya N. Shindyalov, and Philip E. Bourne. The Protein Data Bank. *Nucleic Acids Research*, 28(1):235–242, 2000. doi: 10.1093/nar/28.1.235.
- Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. *Lecture Notes in Computer Science*, 1973:316–330, 2001. doi: 10.1007/3-540-44503-X_20.
- James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi: 10.1561/19000000006.
- E. F. Codd. Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4):397–434, 1979. doi: 10.1145/320107.320109.
- Arthur Dalby, James G. Nourse, W. Douglas Hounshell, Ann K. I. Gushurst, David L. Grier, Burton A. Leland, and John Laufer. Description of several chemical structure file formats used by computer programs developed at molecular design limited. *Journal of Chemical Information and Computer Sciences*, 32(3):244–255, 1992. doi: 10.1021/ci00007a012.
- Daylight Chemical Information Systems. SMARTS — a language for describing molecular patterns, 2019. <https://www.daylight.com/dayhtml/doc/theory/theory.smarts.html>.

- Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017. doi: 10.1038/nbt.3820.
- L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956. doi: 10.4153/CJM-1956-045-5.
- Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and QSAR modeling research. *Journal of Chemical Information and Modeling*, 50(7):1189–1204, 2010. doi: 10.1021/ci100176x.
- Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify II: A practical guide to chemogenomics data curation. *Journal of Chemical Information and Modeling*, 56(7):1243–1252, 2016. doi: 10.1021/acs.jcim.6b00129.
- R. E. Gomory and T. C. Hu. Multi-terminal network flows. *Journal of the Society for Industrial and Applied Mathematics*, 9(4):551–570, 1961. doi: 10.1137/0109047.
- Stephen R. Heller, Alan McNaught, Igor Pletnev, Stephen Stein, and Dmitrii Tchekhovskoi. InChI, the IUPAC international chemical identifier. *Journal of Cheminformatics*, 7:23, 2015. doi: 10.1186/s13321-015-0068-4.
- Tomasz Imieliński and Witold Lipski. Incomplete information in relational databases. *Journal of the ACM*, 31(4):761–791, 1984. doi: 10.1145/1634.1886.
- International Organization for Standardization. ISO/IEC 14977:1996 — information technology: Syntactic metalanguage: Extended BNF, 1996. Standard.
- Craig A. James. OpenSMILES specification. Version 1.0, 2016. <http://opensmiles.org/opensmiles.html>.
- Johannes Köster and Sven Rahmann. Snake-make — a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012. doi: 10.1093/bioinformatics/bts480.
- Mario Krenn, Florian Häse, AkshatKumar Nigam, Pascal Friederich, and Alán Aspuru-Guzik. Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation. *Machine Learning: Science and Technology*, 1(4):045024, 2020. doi: 10.1088/2632-2153/aba947.
- Greg Landrum. RDKit: Open-source cheminformatics, 2024. <https://www.rdkit.org>.
- Leonid Libkin. SQL’s three-valued logic and certain answers. *ACM Transactions on Database Systems*, 41(1):1–28, 2016. doi: 10.1145/2877206.
- Marjan Mernik, Jan Heering, and Anthony M. Sloane. When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4):316–344, 2005. doi: 10.1145/1118890.1118892.
- Luc Moreau and Paul Groth. Provenance: An introduction to PROV. *Synthesis Lectures on the Semantic Web*, 3(4):1–129, 2013. doi: 10.2200/S00528ED1V01Y201308WBE007.
- Peter Murray-Rust and Henry S. Rzepa. CML: Evolution and design. *Journal of Cheminformatics*, 3:44, 2011. doi: 10.1186/1758-2946-3-44.
- Noel M. O’Boyle. Towards a universal SMILES representation — a standard method to generate canonical SMILES based on the InChI. *Journal of Cheminformatics*, 4:22, 2012. doi: 10.1186/1758-2946-4-22.
- Noel M. O’Boyle, Michael Banck, Craig A. James, Chris Morley, Tim Vandermeersch, and Geoffrey R. Hutchison. Open Babel: An open chemical toolbox. *Journal of Cheminformatics*, 3:33, 2011. doi: 10.1186/1758-2946-3-33.
- Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. Semantics and complexity of SPARQL. *ACM Transactions on Database Systems*, 34(3):1–45, 2009. doi: 10.1145/1567274.1567278.
- Mechthild Stoer and Frank Wagner. A simple min-cut algorithm. *Journal of the ACM*, 44(4):585–591, 1997. doi: 10.1145/263867.263872.

Arie van Deursen, Paul Klint, and Joost Visser. Domain-specific languages: An annotated bibliography. *ACM SIGPLAN Notices*, 35(6): 26–36, 2000. doi: 10.1145/352029.352035.

David Weininger. SMILES, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988. doi: 10.1021/ci00057a005.

David Weininger, Arthur Weininger, and Joseph L. Weininger. SMILES. 2. algorithm for generation of unique SMILES notation. *Journal of Chemical Information and Computer Sciences*, 29(2):97–101, 1989. doi: 10.1021/ci00062a008.

John D. Westbrook, Jasmine Y. Young, Chenghua Shao, Zukang Feng, Vladimir Guranovic, Catherine L. Lawson, Brinda Vallat, Paul D. Adams, John M. Berrisford, Gérard Bricogne, et al. PDBx/mmCIF ecosystem: Foundational semantic tools for structural biology. *Journal of Molecular Biology*, 434(11): 167599, 2022. doi: 10.1016/j.jmb.2022.167599.

Antony J. Williams, Sean Ekins, and Valery Tkachenko. Towards a gold standard: Regarding quality in public domain chemistry databases and approaches to improving the situation. *Drug Discovery Today*, 17(13–14):685–701, 2012. doi: 10.1016/j.drudis.2012.02.013.

Egon L. Willighagen, John W. Mayfield, Jonathan Alvarsson, Arvid Berg, Lars Carlsson, Nina Jeliaskova, Stefan Kuhn, Tomáš Pluskal, Miquel Rojas-Chertó, Ola Spjuth, et al. The Chemistry Development Kit (CDK) v2.0: Atom typing, depiction, molecular formulas, and substructure searching. *Journal of Cheminformatics*, 9:33, 2017. doi: 10.1186/s13321-017-0220-4.

Douglas Young, Todd Martin, Raghuraman Venkatapathy, and Paul Harten. Are the chemical structures in your QSAR correct? *QSAR & Combinatorial Science*, 27(11–12):1337–1345, 2008. doi: 10.1002/qsar.200810084.